

# Criptografía — Vulnerability Report (SDDV)

Barrios Aguilar Dulce Michelle  
Caballero Martínez Sergio Jair  
Contreras Colmenero Emilio Sebastian  
Martínez López Evan Emiliano

Facultad de Ingeniería · UNAM 2026-2

## Resumen ejecutivo

Como continuación de la auditoría previa al sistema **SDDV** (Secure Digital Document Vault), donde se confirmó que las cinco capas de defensa contra modificación funcionan correctamente, en esta segunda fase identificamos dos vulnerabilidades reales en el sistema, las reproducimos con scripts ejecutables y aplicamos los parches correspondientes con sus respectivos commits.

### Hallazgos:

| ID       | Título                        | CWE     | Severidad | Commit del fix |
|----------|-------------------------------|---------|-----------|----------------|
| VULN-001 | Path Traversal vía filename   | CWE-22  | HIGH      | b40d0c0        |
| VULN-002 | Replay Attack (sin freshness) | CWE-294 | MEDIUM    | 6a98cd1        |

# VULN-001 — Path Traversal vía filename metadata

## Title

Path Traversal en el campo filename de los contenedores SDDV/SDDH (CWE-22)

## Description

Al cifrar un archivo, el sistema toma el parámetro `filename` provisto por el usuario y lo almacena dentro del AAD (Associated Authenticated Data) del contenedor sin ninguna validación. Al descifrar, el filename se devuelve tal cual en el campo `metadata['filename']`.

Una aplicación cliente que use ese valor para construir una ruta de escritura (patrón habitual: `os.path.join(out_dir, meta['filename'])`) puede ser inducida a escribir el plaintext fuera del directorio destino esperado. Esto se logra incluyendo en el filename componentes como `../`, rutas absolutas (`/etc/...`), null bytes (`\x00`) o caracteres de control.

El problema afecta tanto a `crypto/aead.py` (`encrypt_file` / `decrypt_file`) como a `crypto/hybrid.py` (`encrypt_for_recipients` / `decrypt_for_recipient`), porque ambos comparten la lógica de construcción y parseo del header.

## Impact

Severidad: HIGH (CWE-22)

**CIA Triad:** Integridad (sobrescritura de archivos), potencialmente Disponibilidad y Confidencialidad como consecuencia.

**Vector:** cualquier emisor que pueda cifrar un contenedor (incluso un usuario legítimo del sistema) puede craftear un payload que sobrescriba archivos arbitrarios cuando el destinatario lo descifre con una aplicación que confíe en el filename.

**Consecuencias concretas:**

- Sobrescritura de archivos de configuración (`/etc/cron.d/`, `.bashrc`, `.ssh/authorized_keys`)
- Path traversal hacia binarios del sistema → escalación a RCE
- Sobrescritura de archivos del usuario sin que la aplicación lo advierta
- Ataques de “file write where” en CTFs y entornos de despliegue

## Reproduction steps

### Pre-requisitos:

- Tener instalado el paquete `cryptography` y el código del proyecto.
- Estar en la raíz del repositorio.

**Paso 1.** Verificar el commit ANTERIOR al fix para reproducir la vulnerabilidad:

```
git checkout 867516c # commit antes de VULN-001
```

**Paso 2.** Ejecutar el script de reproducción:

```
PYTHONPATH=. python3 audit/vuln1_path_traversal.py
```

**Paso 3.** Observar el output. El sistema acepta filenames maliciosos sin protestar:

```
[ACEPTADO] filename = '../.../etc/passwd'
           metadata['filename'] retornado = '../.../etc/passwd'
```

```
[ACEPTADO] filename = '/etc/shadow'
           metadata['filename'] retornado = '/etc/shadow'
[ACEPTADO] filename = 'valid.txt\x00../../../../tmp/pwned'
           metadata['filename'] retornado = 'valid.txt\x00../../../../tmp/pwned'
...
```

**Paso 4.** En el CASO 3 del script se ejecuta una explotación concreta:

1. Se crea un directorio destino legítimo /tmp/sddv\_audit\_XXX/.
2. Se crea un “archivo crítico” en otro directorio: /tmp/critical\_YYY/config.cfg con contenido contenido\_legitimo=true.
3. El atacante cifra un payload contenido\_malicioso=true con filename relativo apuntando fuera del directorio destino: ../critical\_YYY/config.cfg.
4. La aplicación vulnerable hace `open(os.path.join(out_dir, meta['filename']), 'wb').write(plaintext)`
5. **Resultado:** el archivo crítico fue sobrescrito por el contenido del atacante.

**Paso 5.** Reproducir manualmente con código mínimo:

```
from crypto.aead import encrypt_file, decrypt_file
container, key = encrypt_file(b"payload", "../../../../etc/passwd")
plaintext, meta = decrypt_file(container, key)
print(meta['filename'])    # imprime '../../../../etc/passwd'
```

## Proposed fix

### Razonamiento:

La librería criptográfica no debe asumir que la aplicación cliente sanitiza el filename. Por el principio de **defense-in-depth**, la validación debe ocurrir tanto al cifrar (para no propagar payloads maliciosos) como al descifrar (para proteger consumidores que usen versiones viejas de la librería para cifrar).

La validación debe rechazar:

- Separadores de path (/ , \)
- Componentes de directorio padre (..)
- Rutas absolutas (Unix y Windows)
- Null bytes (CWE-158)
- Caracteres de control (< 0x20)
- Filename vacío o demasiado largo (>255 bytes)

Se implementó como una función helper `_validate_filename()` en `crypto/aead.py` y se invoca desde:

- `_build_header()` (al cifrar SDDV)
- `_parse_header()` (al descifrar SDDV — defense-in-depth)
- `_build_hybrid_header()` (al cifrar SDDH)
- `_parse_hybrid_header()` (al descifrar SDDH — defense-in-depth)

### Diff del parche (extracto de `crypto/aead.py`):

```
def _validate_filename(filename: str) -> None:
    """Valida que el filename sea seguro para uso en filesystem (CWE-22)."""
    if not isinstance(filename, str):
        raise ValueError("filename debe ser str")
    if not filename:
        raise ValueError("filename no puede estar vacío")
    if len(filename.encode("utf-8")) > MAX_FILENAME_LEN:
        raise ValueError(f"filename excede {MAX_FILENAME_LEN} bytes")
```

```

if "\x00" in filename:
    raise ValueError("filename contiene byte nulo (null byte injection)")
if any(ord(c) < 0x20 for c in filename):
    raise ValueError("filename contiene caracteres de control")
if "/" in filename or "\\" in filename:
    raise ValueError("filename contiene separadores de path")
if filename in (".", ".."):
    raise ValueError("filename no puede ser '.' o '..')")
if len(filename) >= 2 and filename[1] == ":":
    raise ValueError("filename parece ruta absoluta de Windows")
if posixpath.normpath(filename) != filename:
    raise ValueError("filename contiene componentes de path no canonicos")

```

**Commit del parche:** b40d0c0 ([feature/d3-hybrid](#))

#### Cambios:

- crypto/aead.py: +50 líneas (helper + invocaciones)
- crypto/hybrid.py: +5 líneas (import + 2 invocaciones)
- tests/test\_security\_patches.py: +130 líneas (16 tests de regresión)

**Verificación post-parche:** ejecutar `PYTHONPATH=. python3 audit/vuln1_path_traversal.py` después del fix produce `ValueError: filename contiene separadores de path` para todos los payloads maliciosos. Los tests de regresión (`pytest tests/test_security_patches.py::TestVuln001PathTraversal -v`) pasan los 16 casos.

## VULN-002 — Replay Attack (timestamp sin freshness)

### Title

Falta de validación de freshness en timestamp permite replay attacks (CWE-294)

### Description

El sistema almacena un campo `timestamp` (Unix time, 8 bytes big-endian) dentro del AAD del contenedor. Al estar en el AAD, el timestamp queda autenticado: cualquier modificación lo invalida. Sin embargo, las funciones de descifrado (`decrypt_file`, `decrypt_for_recipient`, `secure_verify_and_decrypt`) **nunca comparan el timestamp contra el tiempo actual**.

Esto significa que un contenedor firmado válidamente sigue siendo aceptado como auténtico para siempre. Un atacante con acceso pasivo al canal de comunicación o al storage puede:

1. Capturar un contenedor legítimo firmado por Alice en el momento  $T_0$ .
2. Reproducirlo (sin modificarlo) en el momento  $T_0 + \text{delta-t}$  donde `delta-t` puede ser horas, días o años.
3. Bob lo recibe, verifica la firma Ed25519 (que sigue válida porque los bytes no cambiaron), y procesa el mensaje como si fuera reciente.

**Consideración crítica:** la firma Ed25519 es **determinista** (RFC 8032), así que el atacante no puede modificar el timestamp para falsificar frescura — pero tampoco lo necesita, porque el sistema simplemente no valida la edad del mensaje.

### Impact

Severidad: MEDIUM (CWE-294)

**CIA Triad:** Autenticidad/Frescura. Aunque el sistema preserva la confidencialidad e integridad del mensaje, no garantiza que el mensaje sea reciente.

**Vector:** acceso pasivo (red, storage, backups). El atacante no necesita romper criptografía.

**Consecuencias concretas:**

- Re-ejecución de comandos firmados antiguos (transferencias, autorizaciones, votos)
- Bypass de revocaciones: si Alice revoca su autorización, los mensajes pasados siguen siendo aceptados
- Replay de credenciales o tokens si se transmiten en este formato
- Aceptación de timestamps imposibles (futuro) por bug de cliente o reloj mal configurado

**Severidad limitada por:** el atacante necesita un mensaje legítimo previo (no es ataque “cold”), y el contenido del mensaje no se revela (la confidencialidad sigue intacta).

### Reproduction steps

**Pre-requisitos:** los mismos que VULN-001.

**Paso 1.** Verificar commit anterior al fix:

```
git checkout b40d0c0 # commit con VULN-001 fixed pero VULN-002 todavia presente
```

**Paso 2.** Ejecutar el script de reproducción:

```
PYTHONPATH=. python3 audit/vuln2_replay.py
```

**Paso 3.** Observar los tres casos:

#### Caso A — Replay simple (SDDV):

Alice cifra un mensaje con timestamp del 2020-01-01:  
contenido: b'Transferir \$1000 de Alice a Bob'

Tiempo presente: 2026-05-05

Atacante reentrega el mismo contenedor (sin modificarlo).

Bob descifra:

edad del mensaje: 2317 días

contenido recuperado: b'Transferir \$1000 de Alice a Bob'

[!!! VULNERABILIDAD CONFIRMADA !!!]

El sistema acepto un mensaje cuyo timestamp es de hace mas de 2317 días.

#### Caso B — Replay sobre contenedor firmado D5:

Alice firma una autorización con timestamp de 2024. Bob la recibe en 2026 y `secure_verify_and_decrypt` la acepta sin protestar, devolviendo b'Autorizo la transferencia de fondos. - Alice' como si fuera reciente. **689 días de antigüedad ignorados.**

#### Caso C — Timestamp futuro:

Un cliente con reloj mal configurado o un atacante puede generar contenedores con timestamp del año 2099. El sistema los acepta sin distinguir.

**Paso 4.** Reproducir manualmente con código mínimo:

```
import time
from crypto.aead import encrypt_file, decrypt_file

# Atacante captura contenedor antiguo (timestamp del 2020)
container, key = encrypt_file(b"comando antiguo", "cmd.txt", timestamp=1577836800)

# Sistema lo acepta hoy sin protestar
plaintext, meta = decrypt_file(container, key)
print(plaintext, "edad:", (time.time() - meta['timestamp']) / 86400, "dias")
```

## Proposed fix

### Razonamiento:

El estándar para protección anti-replay en sistemas autenticados es validar el timestamp contra una **ventana de freshness**. Las decisiones de diseño:

1. **Default seguro:** la validación está activa por default con `max_age_seconds = 7` días. La aplicación cliente puede ajustar la ventana o pasar `None` para deshabilitar (útil para herramientas de archive/forense).
2. **Tolerancia a clock skew:** se permite hasta 5 minutos de timestamp en el futuro (por desfase de relojes entre cliente y servidor). Más allá de eso se rechaza.
3. **Sin estado:** la validación es puramente función del timestamp del contenedor y `time.time()`. No requiere mantener un cache de nonces vistos.
4. **Backward compatibility:** `max_age_seconds=None` permite descifrar contenedores legacy sin freshness check, manteniendo la API extensible.

### Diff del parche (extracto de crypto/aead.py):

```
DEFAULT_MAX_AGE = 7 * 24 * 60 * 60    # 7 días por defecto
MAX_FUTURE_SKEW = 5 * 60              # 5 minutos de tolerancia

def _validate_timestamp(timestamp, max_age_seconds):
    if max_age_seconds is None:
        return
    now = int(time.time())
    age = now - timestamp
    if age > max_age_seconds:
        raise ValueError(f"timestamp demasiado antiguo: {age} segundos "
                        f"(max permitido: {max_age_seconds})")
    if age < -MAX_FUTURE_SKEW:
        raise ValueError(f"timestamp en el futuro: {-age} segundos "
                        f"(max skew permitido: {MAX_FUTURE_SKEW})")

def decrypt_file(container, key, max_age_seconds=DEFAULT_MAX_AGE):
    metadata, header_end = _parse_header(container)
    _validate_timestamp(metadata["timestamp"], max_age_seconds)
    # ... resto del flujo de descifrado ...
```

Commit del parche: 6a98cd1 ([feature/d3-hybrid](#))

### Cambios:

- crypto/aead.py: +35 líneas (helper + constantes + parámetro en `decrypt_file`)
- crypto/hybrid.py: +5 líneas (parámetro en `decrypt_for_recipient`)
- crypto/secure\_send.py: +5 líneas (parámetro en `secure_verify_and_decrypt`)
- tests/test\_aead.py, test\_hybrid.py: +2 líneas (pasar `max_age_seconds=None` en tests de roundtrip que usan timestamps fijos del pasado)
- tests/test\_security\_patches.py: +75 líneas (8 tests de regresión)

**Verificación post-parche:** ejecutar `PYTHONPATH=. python3 audit/vuln2_replay.py` después del fix produce `ValueError: timestamp demasiado antiguo: 200177807 segundos (max permitido: 604800)` en el caso A, y excepciones equivalentes en los casos B y C. Los tests de regresión (`pytest tests/test_security_patches.py::TestVuln002ReplayAttack -v`) pasan los 8 casos. La suite completa (146 tests) sigue verde.

## Verificación de los parches

### Tests de regresión

Después de aplicar ambos parches, la suite de tests completa pasa:

```
$ PYTHONPATH=. python3 -m pytest tests/
===== test session starts =====
collected 146 items

tests/test_aead.py ..... [ 18%]
tests/test_d5_hybrid_signed.py ..... [ 38%]
tests/test_hybrid.py ..... [ 59%]
tests/test_keys.py ..... [ 71%]
tests/test_security_patches.py ..... [ 88%]
tests/test_signatures.py ..... [100%]

===== 146 passed in 0.27s =====
```

### Desglose:

- 122 tests pre-existent: siguen pasando (sin regresiones).
- 24 tests nuevos en `test_security_patches.py`:
  - 16 para VULN-001 (path traversal en SDDV, SDDH, defense-in-depth, happy path)
  - 8 para VULN-002 (rechazo de timestamp antiguo, futuro, clock skew, max\_age custom, hybrid, secure\_send)

### Estado del repositorio

```
$ git log --oneline -3
6a98cd1 fix(VULN-002): validar freshness de timestamp para prevenir replay
b40d0c0 fix(VULN-001): validar filename para prevenir path traversal
867516c feat: agregar auditoria de seguridad - script de tampering, log y reporte PDF
```

Ambos commits están en la rama `feature/d3-hybrid` del repositorio público:

- VULN-001: <https://github.com/milliyx/Cryptography/commit/b40d0c0>
- VULN-002: <https://github.com/milliyx/Cryptography/commit/6a98cd1>

## Conclusión

Las dos vulnerabilidades reportadas son representativas de problemas comunes en bibliotecas criptográficas reales:

- **VULN-001 (path traversal)** es un caso clásico de “el cripto no es suficiente”: el sistema garantiza confidencialidad e integridad del contenido, pero la metadata sin sanitizar puede ser usada como vector de ataque cuando la aplicación cliente confía en ella. Es un recordatorio de que **defense-in-depth** debe aplicarse en todas las capas, no solo donde están las primitivas criptográficas.
- **VULN-002 (replay)** ilustra que **autenticidad != frescura**. Una firma válida garantiza “esto fue dicho por X”, pero no “esto fue dicho recientemente”. Cualquier protocolo que dependa de la inmediatez del mensaje debe agregar protección anti-replay explícitamente.



Ambas vulnerabilidades fueron parchadas con cambios mínimos y de bajo riesgo, manteniendo backward compatibility (`max_age_seconds=None`), sin afectar el rendimiento del sistema, y validadas con tests de regresión específicos.